

Lab Practice

Text Cleaning with Regex & Subword Tokenization (BPE)

Course:

NLP

By:

Dr. Zohair Ahmed

Overview

This lab has three parts:

1. **Python re practice**
 - Clean a noisy text file (e.g., tweets or logs)
 - Extract URLs, hashtags, and mentions
2. **Subword tokenization with HuggingFace**
 - Train a small BPE tokenizer on a toy corpus
 - Compare tokenization behavior for rare words under different vocabulary sizes
3. **Short written reflection**
 - Pros/cons of word vs subword tokenization

You can adapt file names, paths, and corpora to your environment.

Part 1 – Regex-Based Text Cleaning in Python

1.1 Setup

- Create a new Python file or notebook, e.g. `NLP_lab_regex_bpe.ipynb`.
- Ensure you have a raw text file, e.g.:
 - `tweets_raw.txt` (one tweet per line), or
 - `logs_raw.txt` (one log entry per line)

You can simulate tweets/logs if you don't have real data.

```
import re
from pathlib import Path

data_path = Path("data")
data_path.mkdir(exist_ok=True)

raw_file = data_path / "tweets_raw.txt"

# Example: create a small synthetic file if you don't have one
if not raw_file.exists():
    sample_lines = [
        "RT @user1: Check out https://example.com !!! #NLP #AI 😊",
        "Error 500 at 2026-02-11T10:23:45Z from 192.168.0.1 - see http://errors.example.org?id=500",
        "New blog post → https://myblog.org/post/123 #blogging #python",
        "@user2 lol that was wild 😂😂 http://bit.ly/xyz",
    ]
    raw_file.write_text("\n".join(sample_lines), encoding="utf-8")

raw_text = raw_file.read_text(encoding="utf-8")
print(raw_text.splitlines()[:3])
```

1.2 Cleaning the Text

Goal: Produce a cleaned version that:

- Normalizes whitespace
- Optionally lowercases
- Replaces URLs, mentions, hashtags, and emojis with placeholders

1.2.1 Define Regex Patterns

```
# URLs: simple pattern (good enough for this lab)
URL_PATTERN = re.compile(r'https?://\S+')

# @mentions: @ followed by letters/digits/underscore
MENTION_PATTERN = re.compile(r'@\w+')

# hashtags: # followed by letters/digits/underscore
HASHTAG_PATTERN = re.compile(r'#\w+')

# emojis / non-ASCII: here we just detect non-ASCII as crude approximation
NON_ASCII_PATTERN = re.compile(r'^\x00-\x7F+')

# collapse multiple spaces
MULTISPACE_PATTERN = re.compile(r'\s+')

```

1.2.2 Apply Cleaning Pipeline

Decide your normalization scheme, e.g.:

- Replace URLs with `<URL>`
- Replace mentions with `<USER>`
- Replace hashtags with `<HASHTAG>`
- Replace non-ASCII (emojis etc.) with `<EMOJI>`
- Lowercase all text
- Strip leading/trailing spaces

```
def clean_line(line: str) -> str:
    line = URL_PATTERN.sub("<URL>", line)
    line = MENTION_PATTERN.sub("<USER>", line)
    line = HASHTAG_PATTERN.sub("<HASHTAG>", line)
    line = NON_ASCII_PATTERN.sub("<EMOJI>", line)
    line = line.lower()
    line = MULTISPACE_PATTERN.sub(" ", line)
    return line.strip()

cleaned_lines = [clean_line(ln) for ln in raw_text.splitlines()]

for original, cleaned in zip(raw_text.splitlines(), cleaned_lines):
    print("ORIG:", original)
    print("CLEAN:", cleaned)
    print("----")

```

1.2.3 Save Cleaned Version

```
clean_file = data_path / "tweets_cleaned.txt"
clean_file.write_text("\n".join(cleaned_lines), encoding="utf-8")
print("Saved cleaned file:", clean_file)

```

1.3 Extract Specific Patterns (URLs, Hashtags, Mentions)

Instead of replacing patterns, now **extract** them for analysis.

1.3.1 Extract from Raw Text

```
lines = raw_text.splitlines()

all_urls = []
all_mentions = []
all_hashtags = []

for ln in lines:
    all_urls.extend(URL_PATTERN.findall(ln))
    all_mentions.extend(MENTION_PATTERN.findall(ln))
    all_hashtags.extend(HASHTAG_PATTERN.findall(ln))

print("URLs:", all_urls)
print("Mentions:", all_mentions)
print("Hashtags:", all_hashtags)
```

1.3.2 Unique & Frequency Counts

```
from collections import Counter

print("Top URLs:", Counter(all_urls).most_common())
print("Top Mentions:", Counter(all_mentions).most_common())
print("Top Hashtags:", Counter(all_hashtags).most_common())
```

Optional extensions:

- Extract timestamps, IP addresses, error codes from logs using regex.
- Use capturing groups to break dates (year, month, day) into structured fields.

Example (IP address pattern, for logs):

```
IP_PATTERN = re.compile(r'\b(?:\d{1,3}\.){3}\d{1,3}\b')

ips = []
for ln in lines:
    ips.extend(IP_PATTERN.findall(ln))

print("IPs:", ips)
```

Part 2 – Subword Tokenization with HuggingFace (BPE)

You can use either:

- `tokenizers` (low-level tokenizer training library), or
- `transformers` (using `AutoTokenizer` with pre-trained models for comparison)

Below is a concrete path using `tokenizers` to **train your own small BPE tokenizer**.

2.1 Install and Import

```
pip install tokenizers
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import Whitespace
from tokenizers.processors import TemplateProcessing
```

2.2 Prepare a Toy Corpus

You can reuse your cleaned tweets/logs plus some extra sentences to emphasize **rare words** and **morphology**.

```
toy_corpus_file = data_path / "toy_corpus.txt"

if not toy_corpus_file.exists():
    extra_sentences = [
        "the water of walden pond is so beautifully clear",
        "low new newer newest lower lowest",
        "subword tokenization helps with rareword rarewords ultra-rareword",
        "renew reset renews renewing renewal",
        "byte pair encoding bpe learns merges for frequent pairs",
        "this corpus is tiny but good enough for demonstration",
    ]
    corpus_text = "\n".join(cleaned_lines + extra_sentences)
    toy_corpus_file.write_text(corpus_text, encoding="utf-8")

print(toy_corpus_file.read_text(encoding="utf-8").splitlines()[:10])
```

2.3 Train a Small BPE Tokenizer (Different Vocabulary Sizes)

We will train **two** BPE tokenizers on the same corpus:

- **Small vocab** (e.g. 200 tokens)
- **Larger vocab** (e.g. 2000 tokens)

2.3.1 Helper Function: Train BPE

```
def train_bpe_tokenizer(corpus_files, vocab_size=200, special_tokens=None):
    if special_tokens is None:
        special_tokens = ["[PAD]", "[UNK]", "[CLS]", "[SEP]", "[MASK]"]
    tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
    tokenizer.pre_tokenizer = Whitespace()

    trainer = BpeTrainer(
        vocab_size=vocab_size,
        special_tokens=special_tokens,
        min_frequency=2, # ignore extremely rare pairs
    )
    tokenizer.train(files=[str(f) for f in corpus_files], trainer=trainer)

    # Optional: add simple post-processor to mimic [CLS] / [SEP] structure
    tokenizer.post_processor = TemplateProcessing(
        single="[CLS] $A [SEP]",
        pair="[CLS] $A [SEP] $B:1 [SEP]:1",
        special_tokens=[
            ("[CLS]", tokenizer.token_to_id("[CLS]")),
            ("[SEP]", tokenizer.token_to_id("[SEP]")),
        ],
    )

    return tokenizer
```

2.3.2 Train Two Tokenizers

```
small_bpe = train_bpe_tokenizer([toy_corpus_file], vocab_size=200)
large_bpe = train_bpe_tokenizer([toy_corpus_file], vocab_size=2000)

print("Small vocab size:", small_bpe.get_vocab_size())
print("Large vocab size:", large_bpe.get_vocab_size())
```

2.4 Compare Tokenization of Rare / Morphologically Rich Words

Choose a small set of test sentences that contain:

- Rare or made-up words
- Morphologically related forms (e.g. **low, lower, lowest, renewal, renewing**)
- Out-of-training-vocabulary style words (e.g. **hyper-antidisestablishmentarian**)

```
test_sentences = [
    "rareword ultrarareword ultra-rareword",
    "renew renews renewing renewal reset",
    "the lower newer lowest newest",
    "hyper-antidisestablishmentarian experiment",
    "multilingual tokenizers sometimes over-segment spanish palabras",
]

def show_encoding(tok, text):
    encoding = tok.encode(text)
    print("Text: ", text)
    print("Tokens:", encoding.tokens)
    print("IDs:   ", encoding.ids)
    print("Len:   ", len(encoding.tokens))
    print()

print("=== SMALL VOCAB BPE ===")
for s in test_sentences:
    show_encoding(small_bpe, s)

print("=== LARGE VOCAB BPE ===")
for s in test_sentences:
    show_encoding(large_bpe, s)
```

What to observe:

- With **small vocabulary**, you should see:
 - Longer token sequences
 - More words broken into character-level or small subword pieces
- With **larger vocabulary**, you should see:
 - More full words represented as single tokens
 - Fewer tokens per sentence

Count total tokens per sentence for each tokenizer:

```
def avg_token_length(tok, sentences):
    lengths = []
    for s in sentences:
        lengths.append(len(tok.encode(s).tokens))
    return sum(lengths) / len(lengths), lengths

small_avg, small_lens = avg_token_length(small_bpe, test_sentences)
large_avg, large_lens = avg_token_length(large_bpe, test_sentences)
```

```
print("Token counts small vocab:", small_lens, "avg =", small_avg)
print("Token counts large vocab:", large_lens, "avg =", large_avg)
```

2.5 Compare to a Pretrained Tokenizer

If you want to compare your BPE to a real LLM tokenizer:

```
pip install transformers
from transformers import AutoTokenizer

gpt_like_tok = AutoTokenizer.from_pretrained("gpt2")

for s in test_sentences:
    toks = gpt_like_tok.tokenize(s)
    print("GPT-2:", s)
    print("Tokens:", toks)
    print("Len:", len(toks))
    print()
```

Note: This uses a large, industrial tokenizer; you don't train it, you only inspect its behavior.

Part 3 – Short Written Reflection

Part 3 – Short Written Reflection (Research-Style Experiment Section Template)

Students should **fill in the placeholders** ([...], (...), { ... }) and can lightly rephrase sentences. The structure and wording is already “paper-like”; they only customize to their run.

Part 3 – Short Written Reflection (Experiment-Style Template)

3.1 Experimental Setup

We [ran / conducted] our tokenization experiments on a [laptop / desktop / cluster] with [CPU: e.g., Intel i7 / Apple M2 / ...], [GPU: none / NVIDIA ...], and [RAM: ... GB]. All code was written in [Python X.Y] using the re library for regular expressions and [HuggingFace tokenizers / transformers] for BPE tokenization.

Our training corpus consisted of [approx. N] lines of [tweets / logs / synthetic sentences]. We [cleaned / normalized] the text by

replacing [URLs] with <URL>, [mentions] with <USER>, [hashtags] with <HASHTAG>, and [non-ASCII / emojis] with <EMOJI>. We [did / did not] lowercase the text and we [did / did not] keep punctuation such as .,! ? as part of the input.

For subword experiments, we [trained] BPE tokenizers on this cleaned corpus using vocabulary sizes of [V₁, e.g. 200] and [V₂, e.g. 2000] tokens, keeping [min_frequency = ...] and [special tokens list] fixed.

3.2 Ablation Study: Vocabulary Size and Tokenization

We compared two BPE models:

- **BPE-Small** with vocabulary size [V₁ = ...]
- **BPE-Large** with vocabulary size [V₂ = ...]

Both models were trained on the **same** corpus and preprocessing. We then tokenized a small test set including:

- Rare / invented words: [e.g., rareword, ultrarareword, hyper-antidisestablishmentarian]
- Morphological families: [e.g., low, lower, lowest, newer, renewal]
- (Optional) Other languages or code-switching: [sentence here, or “not used”]

For each sentence, we recorded:

- Token sequences for BPE-Small and BPE-Large
- The **number of tokens** per sentence under each model

We observed that for our test set, BPE-Small produced token lengths [list example lengths or averages here], while BPE-Large produced [list here]. In general, [brief 1–2 sentence summary: e.g., “BPE-Small gave longer sequences with more splits; BPE-Large gave shorter sequences with more whole-word tokens.”]

3.3 Analysis: Word vs Subword Tokenization (Pros / Cons)

Word-level tokenization (conceptual baseline)

- **Pros (linked to our results)**
 - Intuitive units: one token $\approx$ one word, which would make interpreting outputs like [give one example sentence] straightforward.
 - For our test sentences, a pure word tokenizer would produce about [N_word] tokens per sentence, comparable to or smaller than [BPE-Large].
 - Word units align well with tasks such as [e.g., POS tagging / NER] where labels are defined on words.
- **Cons (linked to our results)**
 - Would require a very large vocabulary for our corpus; by contrast, our subword setups fixed vocab size at [V₁ / V₂].
 - OOV problem: invented forms like [pick one: ultrarareword / made-up hashtag] would be unseen at test time, whereas BPE decomposed them into [describe briefly: e.g., “ultra”, “rare”, “word”].
 - Morphological forms such as [low, lower, lowest / renew, renewal, renewing] would be unrelated at the token level, unlike the shared subpieces we observed with BPE.

Subword tokenization (BPE) in our experiments

- **Pros (empirical)**
 - No true OOVs: all rare / invented forms (e.g. [example]) were representable as sequences of known subwords or characters.
 - Parameter sharing: we observed common subpieces like [e.g., “re”, “new”, “ing”, “est”] reused across [list 2–3 related words], which supports better generalization across morphology.
 - Controllable vocabulary: by setting vocab sizes to [V₁] and [V₂], we directly explored the trade-off between vocabulary size and sequence length.
- **Cons (empirical)**
 - Less interpretable: especially in BPE-Small, some tokens such as [give 1–2 weird subwords from your runs] were not obvious morphemes.
 - Longer sequences: in our results, BPE-Small increased average tokens per sentence from [μ_large] (BPE-Large) to [μ_small], which would increase compute for a transformer.
 - Potential over-segmentation: [if used] our [non-English / code-switched] example was split into [N] tokens vs [M] for comparable English, illustrating the multilingual bias discussed in class.